

Lecture 4

OPTIMIZERS 1: PARAMETRIZATION AND PRECONDITIONING

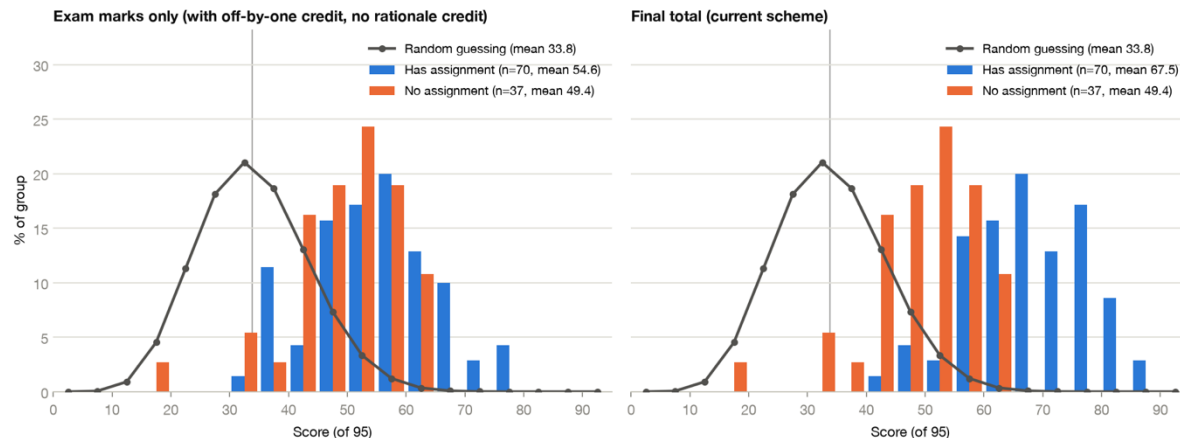
CS312

Tatsu H

Logistics

Assignments and Quiz

- Q1 graded and returned. Regrade requests *in person* at OH



- Don't panic if you see a low number – the median is ~50.
Many more quizzes and we drop 2!
- Doing the assignments are helpful!

Logistics

Assignments and Quiz

Partial credit – what do we consider different types

Criterion 1: Knowing the experiments you've run.

- **Ex A:** Across my 5 seeds in my seed variation experiments, I saw that [..]
- **Ex B:** I saw in my experiments that validation loss could vary [..]
- **None:** [Not citing any assignment or not submitting]

Our intent – Ex A makes it clear that you know what experiments you were actually running. Since Ex B is generic, it makes it difficult to know if there's an actual experiment you know about, or if it's purely your vibes.

We want to reward students who know the experiments they ran in detail

Logistics

Assignments and Quiz

Partial credit – what do we consider different types

Criterion 2: Knowing what the experiments did.

- **Ex A:** I saw that the standard deviation of my run was ~ 0.03 (matches report)
- **Ex B:** I saw that the standard deviation of my run was ~ 0.05 (misremembered report, small error)
- **Ex C:** I saw that the standard deviation of my run was 0.001 (off by many orders of magnitude)
- **Ex C:** I saw that the standard deviation of my run was high (generic)

Our intent – Ex A makes it clear that you know the outcome of the experiments you ran. Ex B and C are different levels of errors in experiment outcomes. At the level of Ex C, it's hard to know if you know your experiments beyond extremely vague generalities.

Logistics

Assignments and Quiz

Partial credit – what do we consider different types

Criterion 3: Relevance and coherence

- **Ex A:** I saw in my seed variation experiment that ..
- **None:** I ran my RMS norm experiment in problem 6 and I remember that the norm value is $1e-5$. So I think the seed variation is..

We didn't see any cases of this in Q1, but there is a loophole that you can kind of cite anything in our current criteria, so we reserve the right to zero out any partial credit that's obviously gaming the system..

We're going to have 3 fields (Citation, Outcome, Rationale) for each of the 3 to make grading more transparent to you.

Logistics

Assignments and Quiz

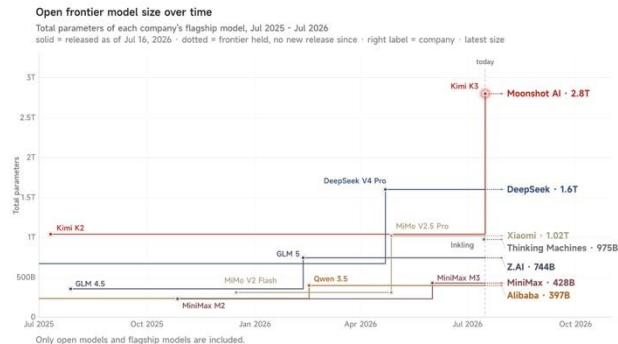
- A2 is due on Thu
There's still time to do a good chunk of A2!
- Q2 will be on Thu
Please make use of partial credit + explicitly cite your experiments

Tutorial scheduling

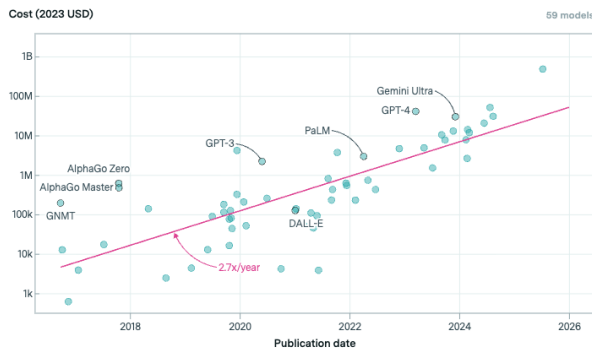
- Emails going out today

Pushing things to the limit

Models are getting larger



The stakes are higher



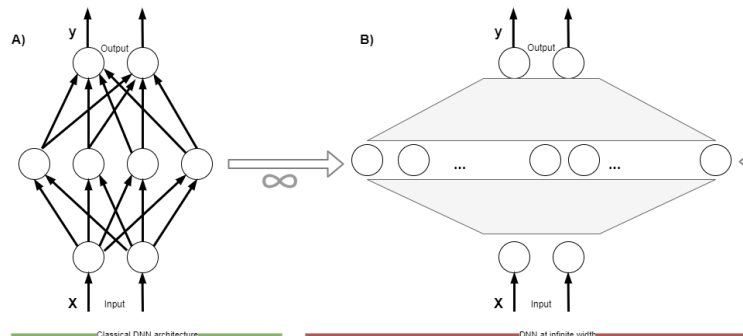


[Subscribe](#) [Sign in](#)

Today's attention often centers on hardware for inference and post-training, yet pre-training a frontier model remains the hardest and most resource-intensive challenge in AI hardware. The TPU platform has passed that test decisively. This stands in sharp contrast to rivals: OpenAI's leading researchers have not completed a successful full-scale pre-training run that was broadly for a new frontier model like GPT-4 in May 2024.

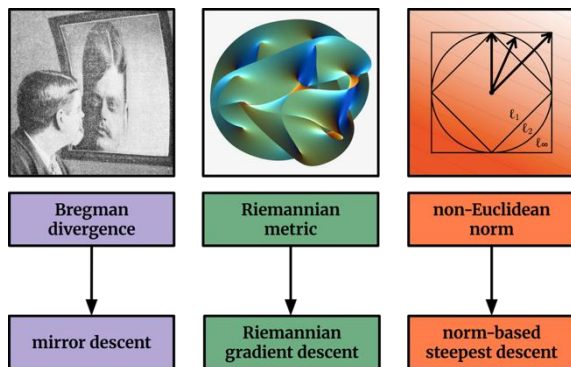
Advanced topics in scaling and norms

Optimization in the infinite model limit



“What’s the maximum step we can take?”

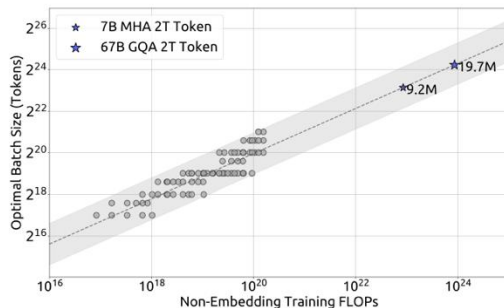
What is the ‘right’ norm for optimization?



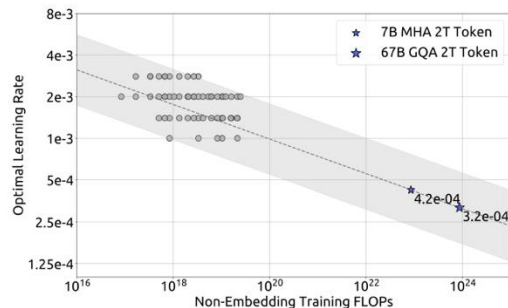
“What step should we take?”

Scaling limits of learning rates

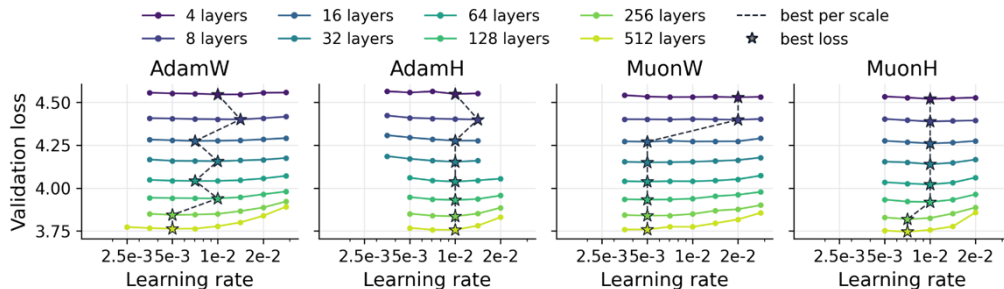
Why do learning rates have to change with model size?



(a) Batch size scaling curve



(b) Learning rate scaling curve



Our game plan: theory (lite)

1. Start with a simple model

Scaling Exponents Across Parameterizations and Optimizers

Katie Everett^{1,2} Lechao Xiao¹ Mitchell Wortsman³ Alexander A. Alemi¹ Roman Novak³ Peter J. Liu¹
Izzeddin Gur¹ Jascha Sohl-Dickstein³ Leslie Pack Kaelbling² Jaehoon Lee¹ Jeffrey Pennington¹

2. Write down our ‘interventions’

3. Study scaling limits of this toy model

A Spectral Condition for Feature Learning

Greg Yang*
xAI

James B. Simon*
UC Berkeley & Imbue

Jeremy Bernstein*
MIT

4. Specialize to the transformers

(also a easy to follow paper)

The simple model

Limit

Width to infinity

Fixed

Depth

Steps

Input norm

Loss gradients

$$\begin{aligned}z_1 &= \phi(n^{-a_1} W_1 \cdot x) \\z_l &= \phi(n^{-a_l} W_l \cdot z_{l-1}), \quad l \in [2, L] \\z_{L+1} &= n^{-a_{L+1}} W_{L+1} \cdot z_L\end{aligned}$$

Why this model?

Simple enough to be tractable, rich enough
to be interesting

The goal: a ‘stable’ limit

As the width goes to infinity, we want 2 key properties to hold

Stable: Activations have constant scale ($z^t = \Theta(1)$) and logits $O(1)$ throughout training

Nontrivial: Change in logits is at least constant scale ($\Delta z = \Omega(1)$) for some timestep

(This rules out the case of zero init or zero learning rate, which are stable but useless)

Aside: feature learning (and why we care)

One well known 'stable' limit: NTK

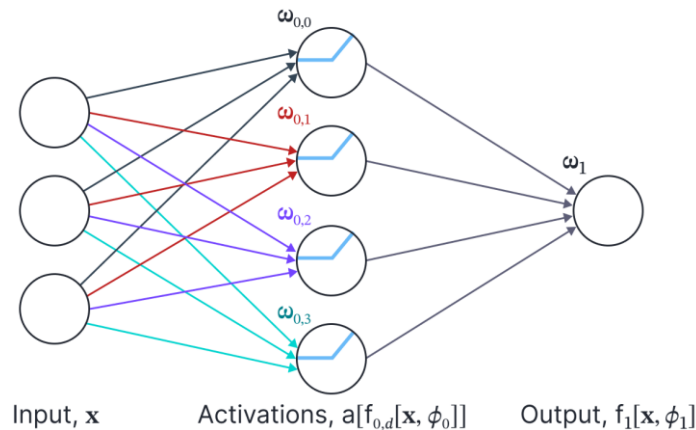
- As we scale up width, keep $a=O(1)$, $w=O(1)$
- Output of layers as $\frac{1}{\sqrt{m}} \sum_i a_i \phi(w_i^\top x)$ ($\frac{1}{\sqrt{m}}$ scaling)
- Keep LR $O(1)$

What happens?

Lots of weights $\rightarrow$ Weights change $O(m^{-\frac{1}{2}})$
The model becomes a 'local' version of init

Reduction to kernels

Roughly equivalent to regression in $\nabla_\theta f(x)$ space



Aside 2: we don't want the NTK limit

The NTK misses a lot of important phenomena..

- “Feature learning” (in the NTK, the first layer is nearly constant)
- In practice, *a lot* worse than the standard NN settings



Figure 1. PCA of Word2Vec embeddings of top US cities and states, for NTK, width-64, and width- ∞ feature learning networks (Definition 4.1). NTK embeddings are essentially random (thus trivializing any “transfer” learning), while cities and states get naturally separated in embedding space as width increases in the feature learning regime.

Table 3. Test Accuracies on Word Analogy after Pretraining with CBOW Word2Vec.

Dataset	number = $\log_2 width$				
	6	8	10	μP	GP/NTK
text8	33.35	41.58	42.56	43.31	0.0
fil9	44.39	54.24	55.69	56.45	0.0

Clearly, stability isn't the be all end all of scaling!

From NTK to μP

Observation: we made various choices about or limit

- Output scale of each block/layer
- Initialization scale
- Learning rate scale

Are there stable and nontrivial limits if we manipulate these variables?

This is the premise of ' μP ' style parametrization methods

ABC parametrization

Recall the formal model: The *abc parametrization* is

- Parameter multiplier: n^{-a_l}
- Parameter initialization: $W_l \sim N(0, n^{-2b_l})$
- Learning rate: $n_l \propto n^{-c_l}$

(Basic assumptions)

- Inputs x are $O(1)$

$$z_1 = \phi(n^{-a_1} W_1 \cdot x)$$

$$z_l = \phi(n^{-a_l} W_l \cdot z_{l-1}), \quad l \in [2, L]$$

$$z_{L+1} = n^{-a_{L+1}} W_{L+1} \cdot z_L$$

$W_1 \in \mathbb{R}^{n \times d}$ for the embedding layer

$W_2, \dots, W_L \in \mathbb{R}^{n \times n}$ for the hidden layers, and

$W_{L+1} \in \mathbb{R}^{d \times n}$ for the readout layer.

We want to find combinations (a,b,c) that lead to stable, nontrivial limits.

Warmup: activation stability

Let's start with something simple we've already looked at *activation stability at init*

Base case:

$$z_1 = \phi(n^{-a_1} W_1 \cdot x)$$

$$\Rightarrow z_1 = O(n^{-a_1 - b_1})$$

(where the big O notation means $-b_1 = \lim_{n \rightarrow \infty} \log_n(\|U\|_{RMS})$)

Argument:

1. We a vector-vector product for each dim, and each product is $O(d)$ with a scaling constant n^{b_1} from init.
2. Thus the 'typical size of an entry' (the RMS norm) is $\sim n^{b_1}$
3. The output scalar is the other exponent term n^{a_1}
4. And we can (mostly) ignore ϕ since its scales things by roughly a constant.

Warmup: activation stability 2

Inductive case: Assume z_{l-1} is RMS $O(1)$. Then, $z_l = O(n^{-a_l-b_l+\frac{1}{2}})$

Argument:

1. We now have a *growing* vector-vector product of dim $O(n)$
2. Inner product of random unit gaussian matrix with a vector is $O(n^{\frac{1}{2}})$
3. Combine with the matrix scale n^{-b_l} and output scale n^{-a_l}

Stability conditions for activation

- $a_1 + b_1 = 0$ (layer 1 has $O(n^{-a_1-b_1})$)
- $a_l + b_l = \frac{1}{2}$ (layer l has $O(n^{-a_l-b_l+\frac{1}{2}})$)
- $a_{L+1} + b_{L+1} \geq \frac{1}{2}$ (we only need layer L readouts to be $O(1)$, not $\Theta(1)$)

Warmup: kaiming init and activation stability

Recall our discussion of the Xavier init

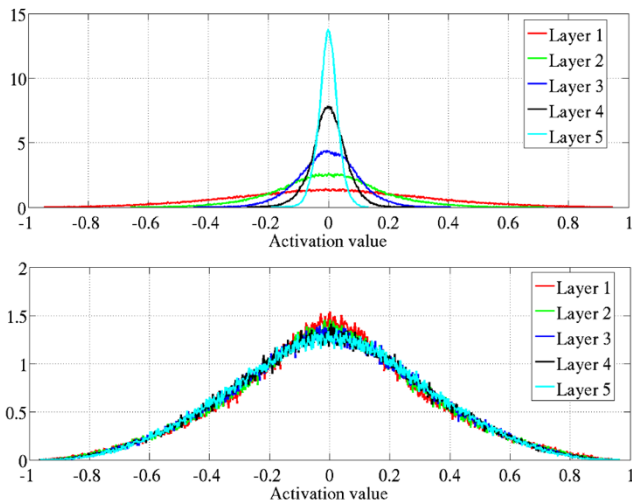


Figure 6: *Activation values normalized histograms with hyperbolic tangent activation, with standard (top) vs normalized initialization (bottom). Top: 0-peak increases for higher layers.*

He Initialization

$$W \sim N\left(0, \sqrt{\frac{2}{D_{in}}}\right)$$

Reminder: Glorot and He initializations keep activation values ‘stable’ across layers at initialization.

No changes to output scale ($a_l = 0$) but ($b_l = 1/2$) from the init scale. Stabilizes the interior and last layers!

Activations are not enough!

Stable: Activations have constant scale ($z^t = \Theta(1)$) and logits $O(1)$ throughout training

Nontrivial: Change in logits is at least constant scale ($z^t = \Omega(1)$) for some timestep

We're missing two big chunks of things:

- Reasoning about the updates ('nontrivial') criterion
- Ensuring this holds throughout training (dynamics)

Gradients at init

Now let's reason about gradients..

Same work, in a backwards way where we chain everything..

$$\frac{\partial \mathcal{L}}{\partial W_l} = \frac{\partial \mathcal{L}}{\partial z_{L+1}} \frac{\partial z_{L+1}}{\partial z_L} \dots \frac{\partial z_{l+1}}{\partial z_l} \frac{\partial z_l}{\partial W_l}$$

Let's assume that our loss function is stable ($\frac{\partial L}{\partial W_{L+1}} = \Theta(1)$).

By the same argument as the *forward* case, $O\left(\frac{\partial L}{\partial W_l}\right) = n^{-g_l}$ where,

$$g_l = \underbrace{a_{L+1} + b_{L+1}}_{\text{Similar logic to 'input layer' from before}} + \underbrace{\left(\sum_{i=l+1} \left(a_i + b_i - \frac{1}{2} \right) \right)}_{\text{Same sqrt(n) calculation (just backwards)}} + \underbrace{a_l}_{\text{No b term since we take grad wrt W}}$$

Additional constraints: $g_l = a_l + a_{L+1} + b_{L+1}$, and $g_{L+1} = a_{L+1}$

Beyond init: reasoning about optimizer behavior

That was the *easy* part. Now to try to work out the whole training process.

This is really hard! We are going to use very aggressive simplifications..

$$\begin{aligned}\text{SGD: } \Delta W_l &= \eta_l \cdot \nabla_{W_l} \mathcal{L} && = O(n^{-g_l - c_l}) \\ \text{Adam: } \Delta W_l &= \eta_l \cdot \frac{\nabla_{W_l} \mathcal{L}}{\|\nabla_{W_l} \mathcal{L}\|} && = O(n^{-c_l})\end{aligned}$$

We will only track coarse grained optimizer behavior.

For SGD, weight updates are roughly gradient sized.

For Adam, updates are actually roughly $O(1)$ (sign of gradient has $\sim O(1)$ RMS)

Tracking changes in updates

Let's track how activations change over time.. At time t ,

$$\begin{aligned} z_l &= n^{-a_l} (W_l + \Delta W_l)(z_{l-1} + \Delta z_{l-1}) \\ &= n^{-a_l} (\underbrace{W_l \cdot z_{l-1}}_{\substack{\text{Init activation} \\ \text{(already controlled)}}} + \underbrace{W_l \Delta z_{l-1} + \Delta W_l \cdot z_{l-1}}_{\substack{\text{Alignment between} \\ \text{updates and init}}} + \underbrace{\Delta W_l \cdot \Delta z_{l-1}}_{\substack{\text{Alignment between} \\ \text{updates}}}) \end{aligned}$$

We will define these alignment exponents as variables

let $r_l := - \lim_{n \rightarrow \infty} \log_n \|\Delta z_l\|$, so that $\Delta z_l \sim n^{-r_l}$.

$$\begin{aligned} \alpha_l &= \lim_{n \rightarrow \infty} \log_n \frac{\|\Delta W_l z_{l-1}\|}{\|\Delta W_l\| \|z_{l-1}\|} & \text{so that } \Delta W_l z_{l-1} &\sim n^{\alpha_l} \|\Delta W_l\| \|z_{l-1}\|, \\ \omega_l &= \lim_{n \rightarrow \infty} \log_n \frac{\|W_l \Delta z_{l-1}\|}{\|W_l\| \|\Delta z_{l-1}\|} & \text{so that } W_l \Delta z_{l-1} &\sim n^{\omega_l} \|W_l\| \|\Delta z_{l-1}\|, \\ u_l &= \lim_{n \rightarrow \infty} \log_n \frac{\|\Delta W_l \Delta z_{l-1}\|}{\|\Delta W_l\| \|\Delta z_{l-1}\|} & \text{so that } \Delta W_l \Delta z_{l-1} &\sim n^{u_l} \|\Delta W_l\| \|\Delta z_{l-1}\|. \end{aligned}$$

Layer by layer..

Let's work out the second step layer by layer.

Layer 1

$$\begin{aligned} z_1^1 &= n^{-a_1}(W_1^0 + \Delta W_1^1)x \\ &= n^{-a_1}W_1^0x + n^{-a_1}\Delta W_1^1x \\ &= z_1^0 + n^{-a_1}\Delta W_1^1x. \end{aligned}$$

We want $z_1^1 = \Theta(1)$ and know $z_1^0 = \Theta(1)$ from init.

The change $n^{-a_1}\Delta W_1^1x$ must be low order ($O(1)$)

Implications

If our optimizer is Adam, $\Delta W_1^1 = O(n^{-c_1})$ and so, $n^{-a_1}\Delta W_1^1x = O(n^{-a_1-c_1})$

Which adds a new constraint of $a_1 + c_1 \geq 0$

Stability during training (intermediate layer)

Layer l

$$\begin{aligned} z_l^1 &= n^{-a_l} W_l^1 z_{l-1}^1 = n^{-a_l} (W_l^0 + \Delta W_l^1) (z_{l-1}^0 + \Delta z_{l-1}^1) \\ &= \underbrace{n^{-a_l} W_l^0 z_{l-1}^0}_{\substack{\text{Init activation} \\ \text{(already controlled)}}} + \underbrace{n^{-a_l} W_l^0 \Delta z_{l-1}^1}_{O(n^{-\frac{1}{2}-r_{l-1}+\omega_l})} + \underbrace{n^{-a_l} \Delta W_l^1 z_{l-1}^0}_{O(n^{-a_l-c_l+\alpha_l})} + \underbrace{n^{-a_l} \Delta W_l^1 \Delta z_{l-1}^1}_{O(n^{-a_l-c_l-r_{l-1}+u_l})} \end{aligned}$$

Implications

If our optimizer is Adam, we have 3 new constraints (one for each term)

- $\frac{1}{2} + r_{l-1} - \omega_l \geq 0$
- $a_l + c_l - \alpha_l \geq 0$
- $a_l + c_l + r_{l-1} - u_l \geq 0$

Stability during training (final logit)

Last layer (logits)

$$\begin{aligned} z_{L+1}^1 &= n^{-a_{L+1}} W_{L+1}^1 z_L^1 = n^{-a_{L+1}} (W_{L+1}^0 + \Delta W_{L+1}^1) (z_L^0 + \Delta z_L^1) \\ &= \underbrace{n^{-a_{L+1}} W_{L+1}^0 z_L^0}_{\text{Init activation (already controlled)}} + \underbrace{n^{-a_{L+1}} W_{L+1}^0 \Delta z_L^1}_{O(n^{-a_{L+1}-b_{L+1}-r_L+\omega_{L+1}})} + \underbrace{n^{-a_{L+1}} \Delta W_{L+1}^1 z_L^0}_{O(n^{-a_{L+1}-c_{L+1}+\alpha_{L+1}})} + \underbrace{n^{-a_{L+1}} \Delta W_{L+1}^1 \Delta z_L^1}_{O(n^{-a_{L+1}-c_{L+1}-r_L+u_{L+1}})} \end{aligned}$$

(these latter two terms are basically the same)

Implications

If our optimizer is Adam, we have 3 constraints (like before)

- $a_{L+1} + b_{L+1} + r_L - \omega_{L+1} \geq 0$
- $a_{L+1} + c_{L+1} - \alpha_{L+1} \geq 0$
- $a_{L+1} + c_{L+1} + r_L - u_{L+1} \geq 0$

Aside: physics math

What am I doing that's kind of concerning (and sketchy)?

Our setup is something like

$$\Theta(n) - O(n) = \Theta(n)$$

But now note that.. We can have something like

$$(n + \log(n)) - n = O(\log(n))$$

So you can have catastrophic cancellation of leading order terms!

We are assuming that the various terms are 'orthogonal enough' to not cancel out.
Some areas of physics use these leading order term calculations

Nontriviality

Nontrivial: Change in logits is at least constant scale ($z^t = \Omega(1)$) for some timestep

Last layer (logits)

$$\begin{aligned} z_{L+1}^1 &= n^{-a_{L+1}} W_{L+1}^1 z_L^1 = n^{-a_{L+1}} (W_{L+1}^0 + \Delta W_{L+1}^1) (z_L^0 + \Delta z_L^1) \\ &= \underbrace{n^{-a_{L+1}} W_{L+1}^0 z_L^0}_{\text{Init activation (already controlled)}} + \underbrace{n^{-a_{L+1}} W_{L+1}^0 \Delta z_L^1}_{O(n^{-a_{L+1}-b_{L+1}-r_L+\omega_{L+1}})} + \underbrace{n^{-a_{L+1}} \Delta W_{L+1}^1 z_L^0}_{O(n^{-a_{L+1}-c_{L+1}+\alpha_{L+1}})} + \underbrace{n^{-a_{L+1}} \Delta W_{L+1}^1 \Delta z_L^1}_{O(n^{-a_{L+1}-c_{L+1}-r_L+u_{L+1}})} \end{aligned}$$

One of the three terms should be at $\Theta(1)$ to ensure the logits change past initialization

Complete constraints

	SGD	Adam	Adafactor
Stability at initialization	$a_1 + b_1 = 0$ $a_l + b_l = 1/2 \text{ for } l \in [2, L]$ $a_{L+1} + b_{L+1} \geq 1/2$		
Stable activations during training	$r_1 := g_1 + a_1 + c_1 \geq 0$ $r_l := \min \begin{cases} g_l + a_l + c_l - \alpha_l \\ g_l + a_l + c_l + r_{l-1} - u_l \\ 1/2 + r_{l-1} - \omega_l \end{cases} \geq 0$ where $g_i := \max(a_{L+1} + b_{L+1}, 2a_{L+1} + c_{L+1}) + a_i$	$r_1 := a_1 + c_1 \geq 0$ $r_l := \min \begin{cases} a_l + c_l - \alpha_l \\ a_l + c_l + r_{l-1} - u_l \\ 1/2 + r_{l-1} - \omega_l \end{cases} \geq 0$	$r_1 := c_1 \geq 0$ $r_l := \min \begin{cases} 1/2 + c_l - \alpha_l \\ 1/2 + c_l + r_{l-1} - u_l \\ 1/2 + r_{l-1} - \omega_l \end{cases} \geq 0$ $c_l \geq 0$
Stable logits during training	$\min \begin{cases} a_{L+1} + b_{L+1} + r_L - \omega_{L+1} \\ 2a_{L+1} + c_{L+1} - \alpha_{L+1} \\ 2a_{L+1} + c_{L+1} + r_L - u_{L+1} \end{cases} \geq 0$	$\min \begin{cases} a_{L+1} + b_{L+1} + r_L - \omega_{L+1} \\ a_{L+1} + c_{L+1} - \alpha_{L+1} \\ a_{L+1} + c_{L+1} + r_L - u_{L+1} \end{cases} \geq 0$	$\min \begin{cases} a_{L+1} + b_{L+1} + r_L - \omega_{L+1} \\ a_{L+1} + b_{L+1} + c_{L+1} - \alpha_{L+1} \\ a_{L+1} + b_{L+1} + c_{L+1} + r_L - u_{L+1} \end{cases} \geq 0$ $c_{L+1} \geq 0$
Nontriviality	$a_{L+1} + b_{L+1} + r_L - \omega_{L+1} = 0$ or $2a_{L+1} + c_{L+1} - \alpha_{L+1} = 0$ or $2a_{L+1} + c_{L+1} + r_L - u_{L+1} = 0$	$a_{L+1} + b_{L+1} + r_L - \omega_{L+1} = 0$ or $a_{L+1} + c_{L+1} - \alpha_{L+1} = 0$ or $a_{L+1} + c_{L+1} + r_L - u_{L+1} = 0$	$a_{L+1} + b_{L+1} + r_L - \omega_{L+1} = 0$ or $a_{L+1} + b_{L+1} + c_{L+1} - \alpha_{L+1} = 0$ or $a_{L+1} + b_{L+1} + c_{L+1} + r_L - u_{L+1} = 0$

This generates a large series of constraints over all of our scaling coefs (a,b,c) and alignment parameters (r, α , ω , u).

Valid parametrizations under the constraints

“b”

“a”

“c”

“c”

		Initialization Variance	Parameter Multiplier	Gradient	SGD LR, Full Align	Adam LR, Full Align	Adafactor LR, Full Align	SGD LR, No Align	Adam LR, No Align	Adafactor LR, No Align
Standard	Embedding	1	1	$1/\sqrt{n}$	$\sqrt{n}$	1	1	$\sqrt{n}$	1	1
	Hidden	$1/n$	1	$1/\sqrt{n}$	$1/\sqrt{n}$	$1/n$	$1/\sqrt{n}$	1	$1/\sqrt{n}$	1
	Readout	$1/n$	1	1	$1/n$	$1/n$	$1/\sqrt{n}$	$1/\sqrt{n}$	$1/\sqrt{n}$	1
NTK	Embedding	1	1	$1/\sqrt{n}$	$\sqrt{n}$	1	1	$\sqrt{n}$	1	1
	Hidden	1	$1/\sqrt{n}$	$1/n$	$\sqrt{n}$	$1/\sqrt{n}$	$1/\sqrt{n}$	n	1	1
	Readout	1	$1/\sqrt{n}$	$1/\sqrt{n}$	1	$1/\sqrt{n}$	$1/\sqrt{n}$	$\sqrt{n}$	1	1
muP	Embedding	$1/n$	$\sqrt{n}$	$1/\sqrt{n}$	1	$1/\sqrt{n}$	1	1	$1/\sqrt{n}$	1
	Hidden	$1/n$	1	$1/n$	1	$1/n$	$1/\sqrt{n}$	$\sqrt{n}$	$1/\sqrt{n}$	1
	Readout	$1/n$	$1/\sqrt{n}$	$1/\sqrt{n}$	1	$1/\sqrt{n}$	1	1	1	1
MFP	Embedding	1	1	$1/n$	n	1	1	n	1	1
	Hidden	1	$1/\sqrt{n}$	$1/n^{1.5}$	n	$1/\sqrt{n}$	$1/\sqrt{n}$	$n^{1.5}$	1	1
	Readout	1	$1/n$	$1/n$	n	1	1	n	$\sqrt{n}$	1

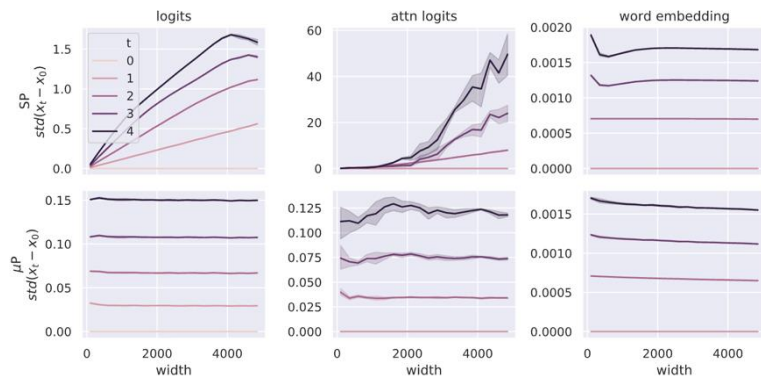
We can pick various alignment parameters (e.g. $\alpha_l = u_l = 1$ or $\alpha_l = u_l = \frac{1}{2}$).

Canonically, $\omega_l = \frac{1}{2}$ for this paper, and we minimize r_l (maximizing feature learning n^{-r_l})

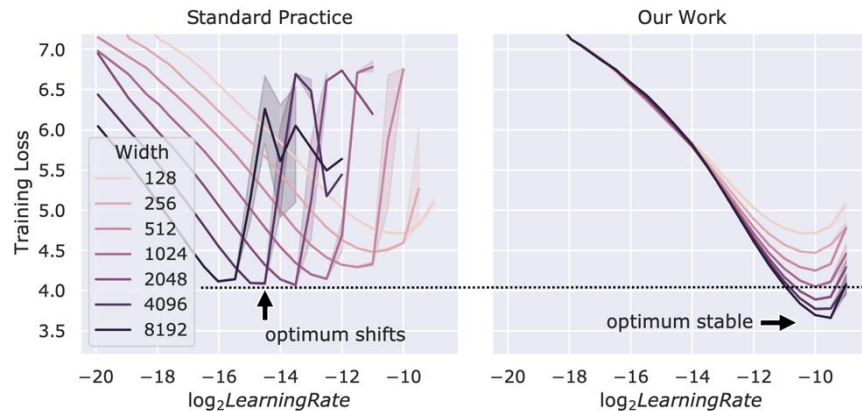
Results from original muP

Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer

Greg Yang^{*×} Edward J. Hu^{*+†} Igor Babuschkin[°] Szymon Sidor[°] Xiaodong Liu[×]
David Farhi[°] Nick Ryder[°] Jakub Pachocki[°] Weizhu Chen[×] Jianfeng Gao[×]
[×]Microsoft Corporation [°]OpenAI



Small-step stability



Learning rate transfer

Alignment in practice

Katie Everett^{1,2} Lechao Xiao¹ Mitchell Wortsman³ Alexander A. Alemi¹ Roman Novak³ Peter J. Liu¹
Izzeddin Gur¹ Jascha Sohl-Dickstein³ Leslie Pack Kaelbling² Jaehoon Lee¹ Jeffrey Pennington¹

Alignment during training varies – between 0.5 and 1.0

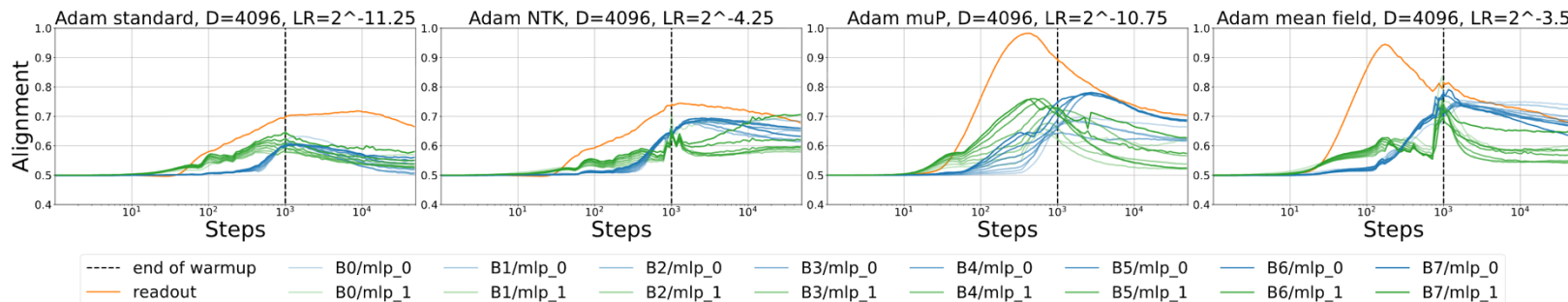


Figure 2. Alignment is intermediate and highly dynamic throughout training, with parameterization-specific patterns. The log alignment ratio metric in readout and hidden (MLP) layers across training steps for each parameterization, for Adam 1.9B parameter models ($H = 32$, $D = 4096$, $B = 256$) using optimal global learning rates. Blue and green curves are for the first and second MLP layers, respectively, in each Transformer block. Transformer blocks are denoted B0 through B7 in the legend. Orange curves are the readout layer.

Per-layer LRs

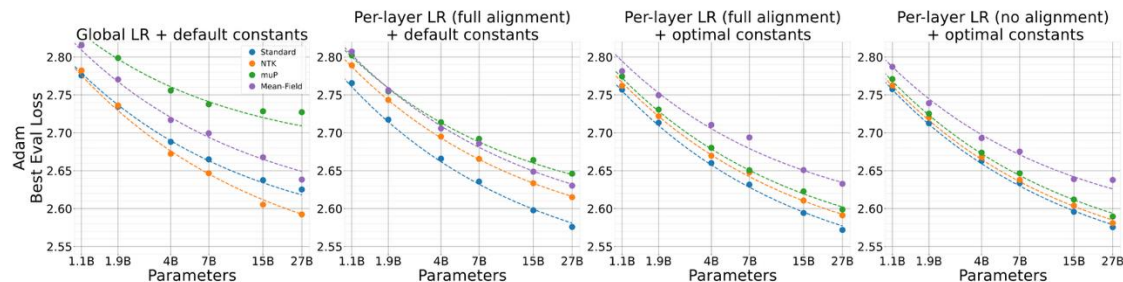
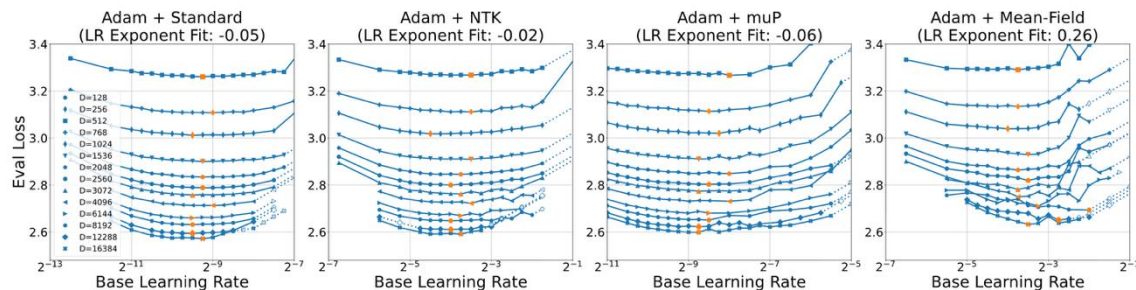


Figure 3. All parameterizations for Adam benefit from per-layer learning rates and tuning per-layer constant learning rate multipliers. Eval loss comparisons for all parameterizations using Adam across a sequence of interventions. From left to right panels: (a) global lr exponents + default constants, (b) per-layer lr exponents assuming full alignment + default constants, (c) per-layer lr exponents assuming full alignment + optimal constants, (d) per-layer lr exponents assuming no alignment + optimal constants.



Per-layer LR dictated by the scaling exponents generally improve performance

Significant gains from respecting the scaling

		Global LR + default constants	Per-layer LR (full align) + default constants	Per-layer LR (full align) + optimal constants	Per-layer LR (no align) + optimal constants
SGD	STP	3.657	3.510	3.385	3.318
	NTK	3.732	3.627	3.313	3.324
	muP	4.224	4.184	3.809	4.222
	MFP	4.092	4.319	4.092	3.898
Adam	STP	2.625	2.576	2.572	2.575
	NTK	2.592	2.615	2.591	2.581
	muP	2.727	2.646	2.599	2.590
	MFP	2.638	2.630	2.633	2.638
Adam+PS	STP	2.580	2.613	2.675	2.577
	NTK	2.570	2.623	2.667	2.566
	muP	2.574	2.655	2.606	2.575
	MFP	2.624	2.640	2.772	2.623

As we go the right and pick the ‘right’ LR scalings we get significant improvements

Recap: muP

We want stability and ‘feature learning’ at once as we scale (width)

Rough order-of-magnitude calculations are very helpful tools

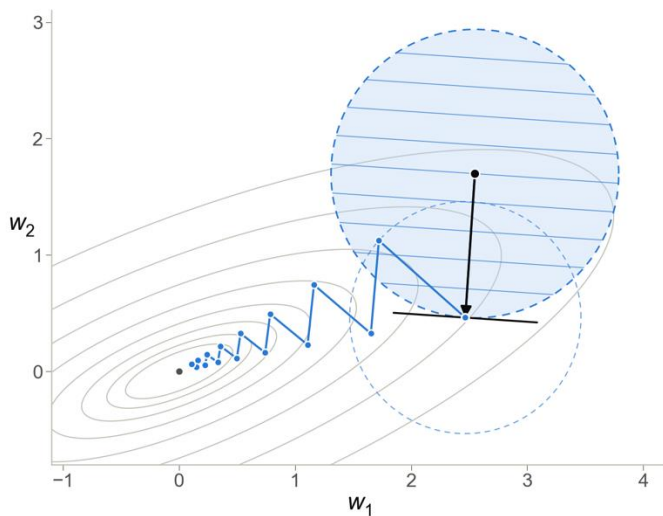
Practical scaling constraints (a,b,c)-parametrization derived from the math

From muP to muon

Story so far: we want activations and logits to be stable and maximal ($\Delta z_l = \Theta(1)$)

For muP: find a series of inits and scalings that roughly enable this

Another view: find parameter updates that naturally follow this

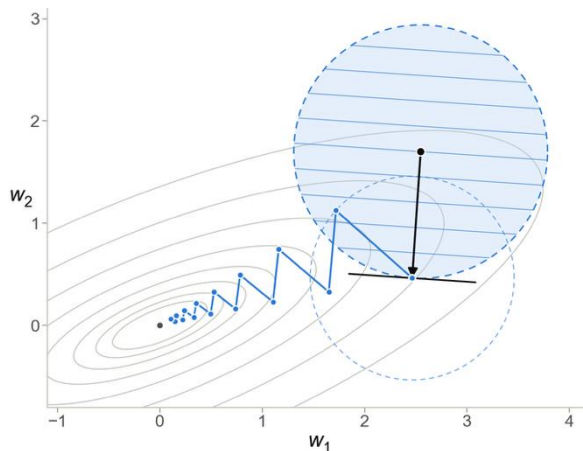


Core idea: what's the right geometry for optimization?

Gradient descent can be viewed as optimizing a linearization within a neighborhood

$$\theta_t = \theta_{t-1} + \arg \min_{\Delta\theta} \left(\langle g_t, \Delta\theta \rangle + \frac{1}{2\eta} \|\Delta\theta\|_2^2 \right)$$

How should we be picking this neighborhood? Why l2?



Neighborhoods from muP style ideas

What does muP have to say about the ‘right’ metric?

$$\begin{aligned} z_{L+1}^1 &= n^{-a_{L+1}} W_{L+1}^1 z_L^1 = n^{-a_{L+1}} (W_{L+1}^0 + \Delta W_{L+1}^1) (z_L^0 + \Delta z_L^1) \\ &= n^{-a_{L+1}} W_{L+1}^0 z_L^0 + n^{-a_{L+1}} W_{L+1}^0 \Delta z_L^1 + n^{-a_{L+1}} \Delta W_{L+1}^1 z_L^0 + n^{-a_{L+1}} \Delta W_{L+1}^1 \Delta z_L^1 \end{aligned}$$

There is actually a term we can very easily control with the optimizer..

$$||n^{-a_l} \Delta W_l z_l|| = n^{-a_l} ||z_l|| \text{ if we have that } \Delta W_l \text{ is an isometry (for square } W).$$

Can we update our parameters via an isometry?

$$\Delta W^t = \arg \min_{||\Delta W||_{op} \leq \rho} \langle G, \Delta W \rangle_F$$

Idealized Muon

By the duality between operator and nuclear norms, for the SVD $G = U\Sigma V^\top$,

$$\Delta W^t = \arg \min_{\|\Delta W\|_{op} \leq \rho} \langle G, \Delta W \rangle_F = -\rho UV^\top$$

This gives rise to what I'd call the 'idealized muon'

Idealized muon. Inputs: scalar LR η , matrix LR ρ

For each matrix valued parameters, set

$$W^t = W^{t-1} + \arg \min_{\|\Delta W\|_{op} \leq \rho} \langle G, \Delta W \rangle_F = W^{t-1} - \rho UV^\top$$

For each scalar valued parameters, set

$$W^t = W^{t-1} + \arg \min_{\|\Delta W\|_2 \leq \eta} \langle G, \Delta W \rangle_F = W^{t-1} - \eta G$$

Practical Muon (newton schultz)

Algorithm 2 Muon

Require: Learning rate η , momentum μ

- 1: Initialize $B_0 \leftarrow 0$
 - 2: **for** $t = 1, \dots$ **do**
 - 3: Compute gradient $G_t \leftarrow \nabla_{\theta} \mathcal{L}_t(\theta_{t-1})$
 - 4: $B_t \leftarrow \mu B_{t-1} + G_t$
 - 5: $O_t \leftarrow \text{NewtonSchulz5}(B_t)$
 - 6: Update parameters $\theta_t \leftarrow \theta_{t-1} - \eta O_t$
 - 7: **end for**
 - 8: **return** θ_t
-

Changes from idealized Muon

1. Use momentum B_t , not the gradient
2. Use NewtonSchultz5, not SVD + sign
3. Use AdamW (or some other advanced optimizer) for the other parameters, not SGD.

What's Newton Schultz..?

Systems concern: Doing the SVD is very slow. Newton-Schultz is a fast, loose approx.

Core trick: Instead of the SVD, consider $G \rightarrow G(G^\top G)^{-\frac{1}{2}} = UV^\top$.

How do we compute $(G^\top G)^{-\frac{1}{2}}$ efficiently? (3rd order) Newton-Schultz is the following eq.

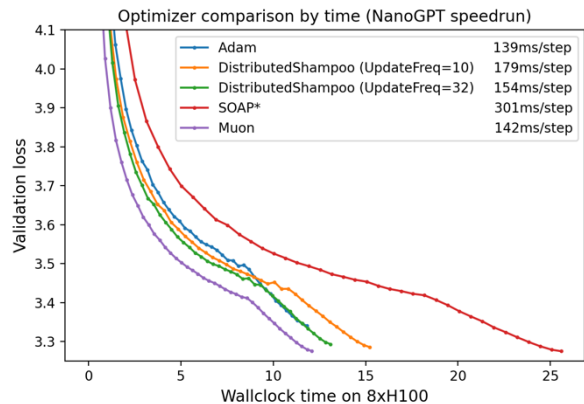
$$Y_{k+1} = \frac{1}{2} Y_k (3I - (G^\top G) Y_k^2)$$

Once we have $(G^\top G)^{-\frac{1}{2}}$, multiply once by G to get what we want. It's all matmuls!

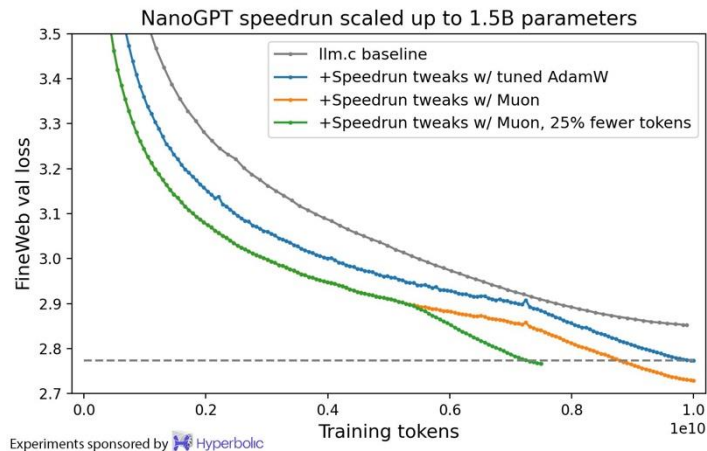
The 'real' Muon uses the quintic expansion with nonstandard coefficients (but the idea is the same)

$$Y_{k+1} = Y_k \left[aI - b(G^\top G) Y_k^2 + c((G^\top G) Y_k^2)^2 \right]$$

From nanoGPT to scale



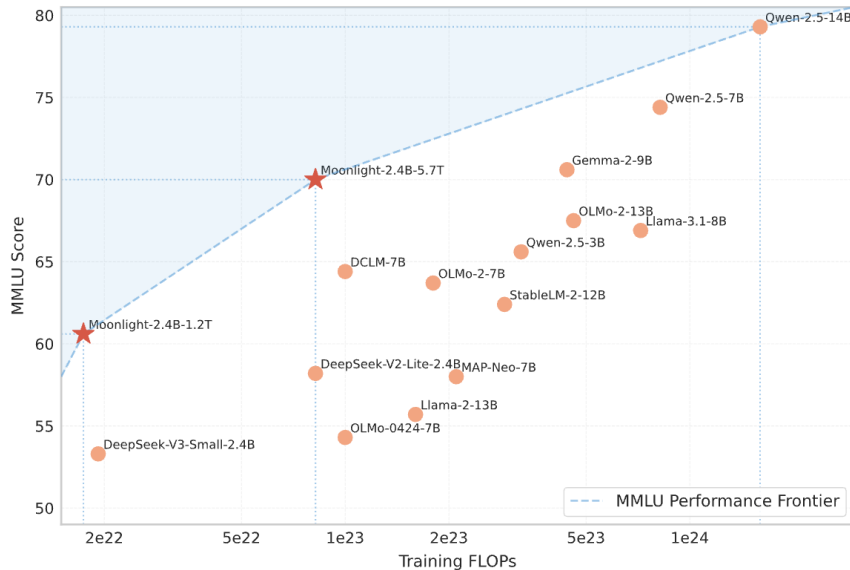
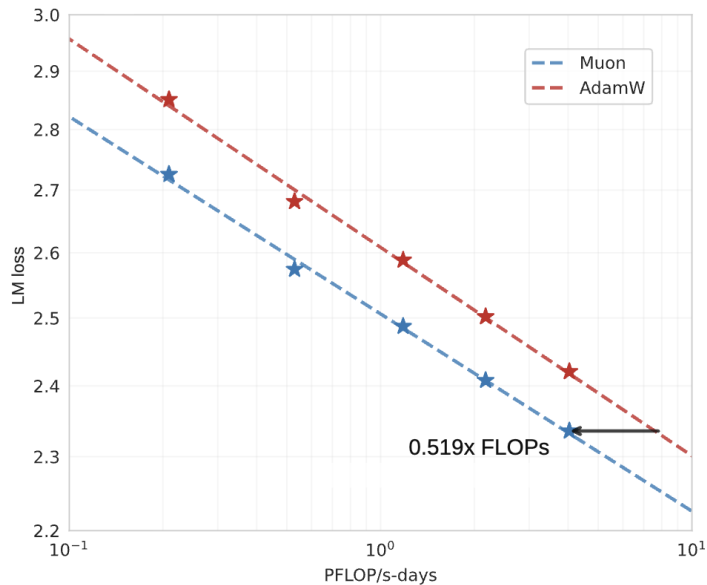
*SOAP is under active development. Future versions will significantly improve the wallclock overhead.



Original Jordan keller blog: clear wins on nanoGPT + scale up experiments

Jingyuan Liu¹ Jianlin Su¹ Xingcheng Yao² Zhejun Jiang¹ Guokun Lai¹ Yulun Du¹
 Yidao Qin¹ Weixin Xu¹ Enzhe Lu¹ Junjie Yan¹ Yanru Chen¹ Huabin Zheng¹
 Yibo Liu¹ Shaowei Liu¹ Bohong Yin¹ Weiran He¹ Han Zhu¹ Yuzhi Wang¹
 Jianzhou Wang¹ Mengnan Dong¹ Zheng Zhang¹ Yongsheng Kang¹ Hao Zhang¹
 Xinran Xu¹ Yutao Zhang¹ Yuxin Wu² Xinyu Zhou¹ Zhilin Yang²

¹ Moonshot AI ² UCLA



Muon works at scale

Stability patches for muon (kimi)

Kimi's muon implementation has a few practical patches

- Weight decay** (adding the usual AdamW decay into the muon update)
- Consistent RMS updates** (scale by $\sqrt{\max(A, B)}$ for rectangular matrices)
...and make it match AdamW's average step size (x0.2)

Table 1: Controlling Muon's Update RMS Across Different Model Params

Methods	Training loss	Validation loss	query weight RMS	MLP weight RMS
Baseline	2.734	2.812	3.586e-2	2.52e-2
Update Norm	2.72	2.789	4.918e-2	5.01e-2
Adjusted LR	2.721	2.789	3.496e-2	4.89e-2

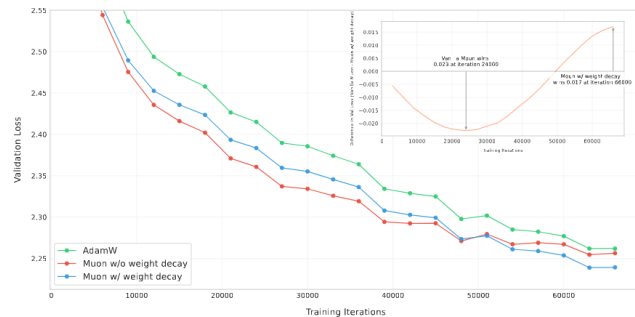


Figure 2: Validation loss curves for AdamW (green), Muon without weight decay (red), and Muon with weight decay (blue).

(And QK-clip, in the actual K2 report)

Why does muon work

Why is it good to orthogonalize the update?

We would first like to observe that one valid answer would be: It just is OK? (Shazeer 2020)

~~to implement, and hence, opposes, computational drawbacks.~~ We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.

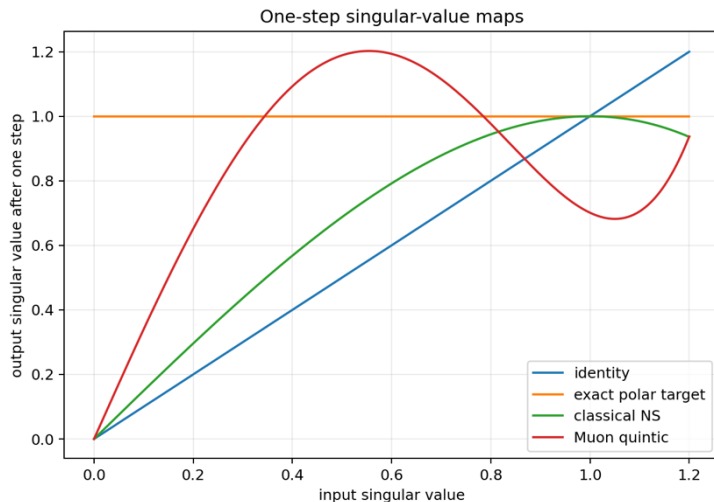
But, for a theoretically-flavored motivation descending from Bernstein & Newhouse (2024)'s analysis of Shampoo (Gupta et al. 2018), see the relationship to Shampoo section.

And for an empirically-flavored motivation, we observe that based on manual inspection, the updates produced by both SGD-momentum and Adam for the 2D parameters in transformer-based neural networks typically have very high condition number. That is, they are almost low-rank matrices, with the updates for all neurons being dominated by just a few directions. We speculate that orthogonalization effectively increases the scale of other "rare directions" which have small magnitude in the update but are nevertheless important for learning.

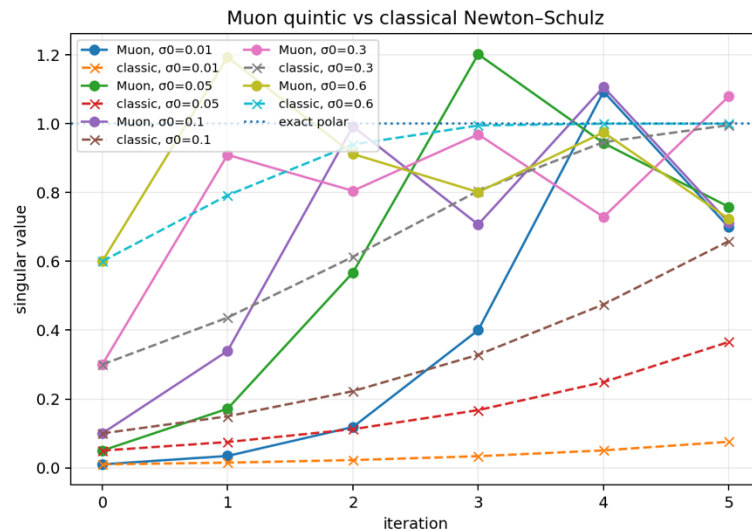
Our muP argument is one (this is the Bernstein and Newhouse argument), but also there's some empirical tests you can do about the 'update rank'

Impact of netwon schultz

Some interesting facts about muon..



The 'coefficients' do not have the right fixed point



But it's fine – fast approx. in 5 steps

Questioning muon

Delving into Muon and Beyond: Deep Analysis and Extensions

Xianbiao Qi^{*1} Marco Chen^{*2} Jiaquan Ye¹ Yelin He¹ Rong Xiao^{†1}

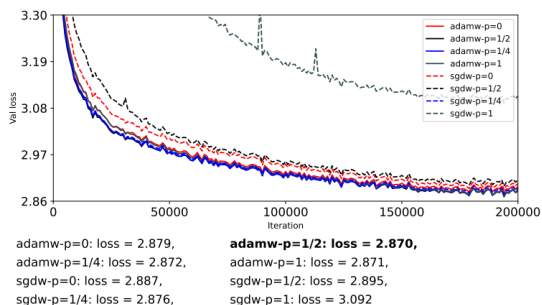
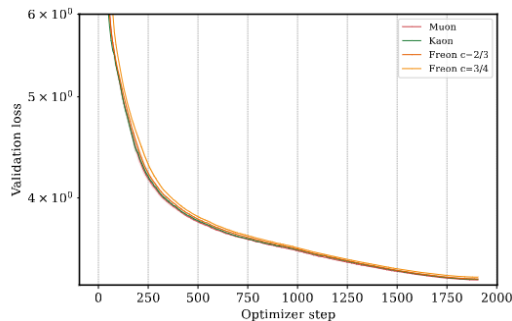
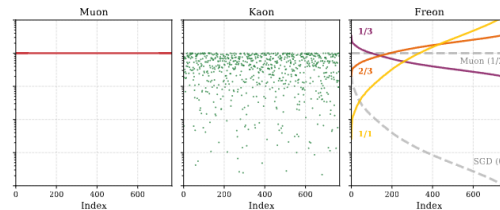


Figure 4. Comparison of four spectral exponents $p \in \{0, \frac{1}{4}, \frac{1}{2}, 1\}$ applied to either the first-moment momentum M_t or the RMS-normalized update $M_t \otimes \sqrt{V}_t$, each shown at its tuned learning rate. Dashed curves correspond to momentum-input variants, while solid curves correspond to RMS-normalized-input variants.

Muon is Not That Special: Random or Inverted Spectra Work Just as Well

Zakhar Shumaylov¹ Nathael Da Costa² Peter Zaika¹ Bálint Mucsányi² Alex Massucco¹
Yoav Gelberg¹ Carola-Bibiane Schönlieb¹ Yarin Gal³ Philipp Hennig²

¹University of Cambridge ²University of Tübingen ³University of Oxford



More evidence that the precise spectrum is not important (and uniform isn't optimal)

Recap: Muon

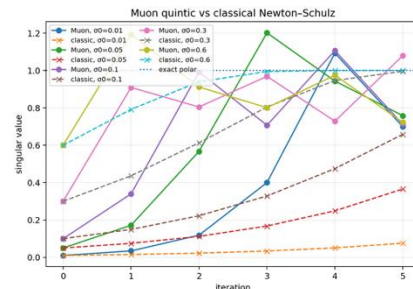
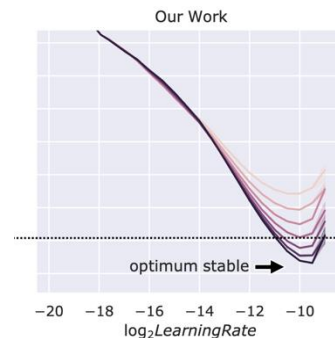
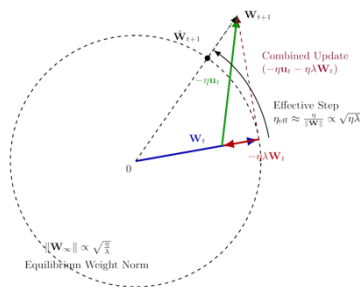
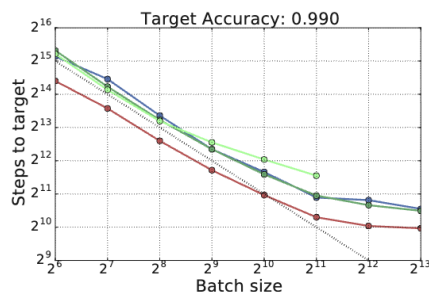
MuP's “update constraints” give natural ideas for a new type of optimizer update

Polar decompositions can be practical (and effective) with newton-schultz

But maybe the ‘unit singular value’ story isn’t quite 100% right

Recap: Optimizers 1

Optimization is complex! A complicated interplay of (light) theory and empirics



Some lessons..

1. What are the 'core phenomena'? (norms, feature update, singular values)
2. What's the right model? (NQM, norm-invariant loss, scaling limits)
3. What's the limits of the theory? (e.g., NQM vs Muon)